

Layer 1: Protocol Parsing (turning raw bytes into structured data)

`src/terminal/tmux/control.zig`

This is the foundational file of the entire tmux control mode feature. Its job is to take a stream of raw bytes coming from a tmux process and turn them into structured, typed values that the rest of Ghostty can reason about.

When tmux runs in control mode, it does not draw a visual interface. Instead, it sends plain-text messages over its standard output. Every message starts with a percent sign. For example, when a shell running inside tmux produces output, tmux sends a message that starts with `percent-output`, followed by a pane identifier and the escaped output data. When a new window is created, tmux sends `percent-window-add` followed by the window identifier. These messages arrive as a continuous stream of bytes mixed in with other terminal data.

The file defines two central types. The first is `Parser`, a state machine with four possible states: `idle` (waiting for a new message to begin), `notification` (accumulating bytes of a message that started with a percent sign), `block` (accumulating the payload of a begin-end command response block), and `broken` (a terminal error state where the parser has encountered something it cannot recover from). The parser processes one byte at a time through a method called `put`. Each call to `put` may return a `Notification` value, or it may return nothing if the byte is part of an incomplete message.

The second central type is `Notification`, which is a tagged union. A tagged union in Zig is like a discriminated variant type: it can hold exactly one of several possible values, and the tag tells you which one. The `Notification` union has twelve variants: `enter` (control mode has started), `exit` (control mode is ending), `block_end` (a command response block completed successfully), `block_err` (a command response block completed with an error), `output` (a pane produced terminal output), `session_changed` (the attached tmux session changed), `sessions_changed` (a session was created or destroyed), `layout_change` (the pane layout within a window changed), `window_add` (a new window was added), `window_renamed` (a window's name changed), `window_pane_changed` (the active pane within a window changed), `client_detached` (another client disconnected), and `client_session_changed` (another client switched sessions).

The parser includes careful validation of what tmux calls guard lines. When you send a command to tmux in control mode, the response comes wrapped between a `percent-begin` line and a matching `percent-end` or `percent-error` line. Each guard line contains a timestamp, a command number, and a flags field. The parser must verify that a line inside a block that happens to start with `percent-end` is actually a guard line and not just coincidental output. This was the subject of a real bug in Ghostty, issue number 11395, where shell output containing the literal text `percent-end` caused the parser to prematurely terminate a command block. The fix requires checking the full guard-line format: exact token count, and all metadata fields must be valid integers.

The parser uses Oniguruma regular expressions to parse individual notification lines. This is functional but is a potential optimization target, since simpler string splitting would suffice for most notification formats.

There are 26 unit tests in this file, covering all implemented notification types, block parsing, guard-line validation edge cases, and carriage return handling.

This file matters for the integration because it is the very first thing that touches tmux data. Every single piece of information Ghostty knows about a tmux session begins here. If a notification type is missing from the Notification union, no higher layer can react to it. Currently, ten notification types are missing from the union, including percent-window-close (essential for closing tabs when tmux windows are destroyed), percent-pause and percent-continue (needed for flow control), and percent-extended-output (the flow-control-aware variant of output).

src/terminal/tmux/layout.zig

This file is a sub-parser dedicated to a single, specific data format: tmux layout strings. When tmux tells Ghostty about the arrangement of panes within a window, it does so using a compact string notation. A single pane might be described as 80x24,0,0,42, meaning 80 columns wide, 24 rows tall, positioned at column zero and row zero, with pane identifier 42. When panes are split, curly braces denote horizontal splits (side by side) and square brackets denote vertical splits (stacked). These can nest to arbitrary depth.

The file defines a Layout type that represents a tree node. Each node has a width, height, x position, and y position. The content of a node is either a pane (a leaf node with a pane identifier), a horizontal container (with an array of child Layout nodes arranged side by side), or a vertical container (with an array of child Layout nodes stacked on top of each other).

The file also implements CRC16 checksum validation. Every layout string from tmux is prefixed with a four-character hexadecimal checksum. The parser can validate this checksum using tmux's specific rotate-right CRC algorithm to ensure the layout string was not corrupted in transit.

There are 31 unit tests covering single panes, both split orientations, arbitrary nesting, every syntax error case, and checksum validation against known tmux checksums.

This file matters for the integration because layout strings are how Ghostty knows where to put native splits. When a user resizes their terminal, splits a pane, or closes a pane, tmux sends a percent-layout-change notification containing a new layout string. Ghostty must parse this string to determine how many panes exist, what their dimensions are, and how they are arranged, then update the native GUI to match.

src/terminal/tmux/output.zig

This file handles the other direction of command communication: when Ghostty sends a command to tmux (like list-windows or list-panes) and receives a response, this file parses that response into typed Zig structs.

The central mechanism is a compile-time code generation system built around tmux format variables. tmux has a rich set of format variables that let you request specific pieces of information. For example, cursor_x gives the cursor column position, pane_in_mode tells you whether copy mode is active, and mouse_any_flag tells you whether the terminal is in any-event mouse tracking mode. The file defines a Variable enum with 31 of these format variables.

The clever part is a compile-time function called FormatStruct. You give it an array of Variable values, and at compile time it generates a Zig struct type with a field for each variable, using the correct Zig type (boolean for flags, unsigned integer for positions and identifiers, string for things like version numbers and layout strings). A companion function called comptimeFormat generates the corresponding tmux format string at compile time, and parseFormatStruct parses delimited text output into the generated struct at runtime.

There are 41 tests covering every variable type, format string generation, struct parsing, delimiter handling, and error cases.

This file matters for the integration because it is how Ghostty extracts structured information from tmux command responses. When the Viewer (described next) sends list-windows to tmux, it gets back a line of delimited text. This file turns that text into a struct with typed fields for session identifier, window identifier, width, height, and layout string. Without this file, the Viewer would have to do ad-hoc string parsing for every command response.

src/terminal/dcs.zig

This file sits at the boundary between Ghostty's general-purpose terminal escape sequence parser and the tmux-specific code. DCS stands for Device Control String, which is a category of terminal escape sequence. When tmux starts control mode with the double-C flag, it sends a DCS sequence with parameter 1000 and final byte p. This is the signal that tells a terminal emulator that tmux control mode has begun.

The file defines a Handler struct that participates in Ghostty's broader DCS handling infrastructure. The key function is tryHook, which examines incoming DCS parameters. When it sees parameter 1000 with final byte p, it creates an instance of the tmux control Parser (from control.zig) and returns an enter notification. From that point forward, every subsequent byte that arrives within the DCS sequence is routed to the tmux Parser via the put method.

When the DCS sequence terminates (signaled by the String Terminator escape sequence), the handler tears down the tmux Parser and returns an exit notification.

There are 5 tests covering tmux entry detection and implicit exit.

This file matters for the integration because it is the detection mechanism. Without it, Ghostty would not know that a tmux control mode session has started. It is the gateway that routes the raw byte stream from the general VT parser into the tmux-specific parsing pipeline.

Layer 2: Semantics (turning parsed data into application state)

src/terminal/tmux/viewer.zig

This is the largest and most complex file in the tmux subsystem, at roughly 2,284 lines. If control.zig is the ears of the tmux integration (it listens to raw protocol data), viewer.zig is the brain. It takes the stream of Notification values from the parser and orchestrates the entire tmux control mode session: discovering windows and panes, populating their content, tracking state changes, and telling the rest of Ghostty what to do.

The file defines a Viewer struct that is a state machine with four states: startup_block (waiting for the initial empty command block that tmux sends when control mode begins), startup_session (waiting for the session-changed notification that tells Ghostty which tmux session it is attached to), command_queue (the main operating state where the Viewer sends commands to tmux and processes responses), and defunct (an unrecoverable error state).

The Viewer maintains several important collections. It has a list of Window structs, each containing a window identifier, dimensions, and a parsed Layout tree. It has a hash map of Pane structs keyed by pane

identifier, where each Pane contains a full Terminal instance. A Terminal in Ghostty is a complete terminal emulator: it has a screen buffer, cursor state, mode flags, scrollback history, and a VT sequence parser. The Viewer creates one of these for every tmux pane, so each pane's content can be independently rendered.

The Viewer's main interface is the next method, which takes an Input (currently always a Notification from the tmux parser) and returns an optional Action. An Action is a tagged union with three variants: exit (the session is ending), command (a string that should be sent to tmux's standard input), and windows (an array of Window structs representing the current state of all windows and panes).

The lifecycle works as follows. After entering control mode, the Viewer waits for the initial block and session-changed notification. It then enters the command queue state and begins sending commands to tmux one at a time, waiting for each response before sending the next. The first command queries the tmux version. The second queries all windows using list-windows with a format string. When the window list response arrives, the Viewer parses it, creates or updates its internal window and pane structures (a process called syncLayouts), and then queues capture commands for each pane. There are four capture commands per pane: primary screen scrollback, primary screen visible area, alternate screen scrollback, and alternate screen visible area. After all captures complete, a list-panes command retrieves terminal state (cursor position, cursor shape, terminal modes, mouse modes, scroll region, tab stops) which is then applied to each pane's Terminal instance.

Once initialization completes, the Viewer enters steady state. Live percent-output notifications are decoded (octal escape sequences are converted back to raw bytes) and fed into the appropriate pane's Terminal via its VT stream parser. Layout change notifications trigger re-parsing of the layout tree, synchronization of panes (creating new ones, pruning removed ones), and capture command sequences for any newly appeared panes. Window-add notifications trigger a full re-query of the window list. Session-changed notifications cause a full reset: all state is discarded and the startup sequence begins again.

There are 8 integration tests using a TestStep framework that lets you construct sequences of tmux notifications and verify the Viewer's resulting actions and internal state.

This file matters for the integration because it is the central coordinator. It owns the definitive state of what tmux windows and panes exist, what their content is, and what their terminal state is. The GUI glue layer that does not yet exist will need to read from the Viewer's Window and Pane structures to create native tabs and splits. The Viewer is already complete enough for basic operation; the challenge is connecting its output to the rest of Ghostty.

The file has several documented limitations. It does not handle percent-window-close because that notification is not in the parser's Notification union. It does not track which pane is active or which window is the current window. It does not handle resize events or user-initiated actions like creating new windows or sending keystrokes. It does not implement flow control. These are all gaps that will need to be filled for a complete integration, but they are secondary to the primary gap: nothing connects the Viewer's internal state to visible GUI elements.

Layer 3: Module Wiring and Build Configuration

src/terminal/tmux.zig

This is a small module re-export file, roughly 14 lines. Its purpose is to provide a clean public interface for the tmux subsystem. It imports the four implementation files (`control.zig`, `layout.zig`, `output.zig`, and `viewer.zig`) and re-exports their key types under convenient names. For example, it exports `control.Parser` as `ControlParser`, `control.Notification` as `ControlNotification`, `layout.Layout` as `Layout`, and the `Viewer` type from `viewer.zig`.

Other parts of Ghostty that need to work with tmux types import them through this module rather than reaching into the individual implementation files. This provides a single point of control for the tmux subsystem's public API.

This file matters for the integration because any new types added to the tmux subsystem (for example, a new `Backend` type for tmux-backed surfaces) would need to be re-exported here so that the rest of Ghostty can access them.

`src/terminal/main.zig`

This is the main module file for the entire terminal subsystem. Among many other exports, it contains a conditional import of the tmux module. The tmux module is only available when the build option `tmux_control_mode` is true. When it is false, the tmux export is set to an empty struct, which means any code that references `terminal.tmux` will compile but have no actual tmux types available.

This file matters for the integration because it is where the tmux feature is gated at the module level. If the integration adds new tmux-related types or modules, they need to be accessible through this file's exports.

`src/terminal/build_options.zig`

This file defines build-time configuration options for the terminal module. The relevant line ties the `tmux_control_mode` option to Oniguruma availability. Oniguruma is a regular expression library that the tmux control parser uses to parse notification lines. In standard Ghostty builds, Oniguruma is available, so `tmux_control_mode` is true.

This file matters for the integration because it determines whether tmux support is compiled in. If the integration were to remove the Oniguruma dependency (for example, by replacing regex parsing with simpler string operations in `control.zig`), this build option would need to be updated to reflect the new dependency.

Layer 4: Integration Glue (connecting parsing to the application)

`src/termio/stream_handler.zig`

This file is the most critical integration point for the tmux feature, and it contains the single most important gap in the current implementation. The stream handler sits between the terminal's escape sequence parser and the rest of the application. When the DCS handler in `dcs.zig` produces tmux notifications, they flow into this file's `dcsCommand` method.

The `dcsCommand` method has a switch statement that handles tmux notifications. When an enter notification arrives, it creates a `Viewer` instance and stores it. When an exit notification arrives, it destroys the `Viewer`. For all other notifications, it feeds them to the `Viewer`'s `next` method and then processes the resulting actions.

For command actions, the stream handler works correctly: it takes the command string from the Viewer and queues it as a write request to the tmux process's standard input. This is how Ghostty sends commands like `list-windows` and `capture-pane` back to tmux.

For the windows action, however, there is a TODO comment and nothing else. This is the exact point where the Viewer says "here are the tmux windows and panes, create the GUI for them" and the stream handler does nothing. This is the primary gap that must be filled for the integration to produce any visible result.

The stream handler also contains existing logging infrastructure. There are `log.info` calls that report tmux control mode events and Viewer actions. These are valuable for debugging and for verifying that the pipeline is working correctly before the GUI glue is implemented.

This file matters for the integration because it is the bridge between the tmux subsystem and the application. The TODO at the windows action handler is literally the point where implementation must begin. The stream handler will need to translate the Viewer's windows action into application runtime (apprt) operations: creating tabs, creating splits, and establishing the communication channels between tmux-backed surfaces and the Viewer.

`src/termio/backend.zig`

This file defines the backend abstraction for terminal input and output. A backend is what actually provides data to a terminal surface and accepts input from it. Currently, the file defines a `Kind` enum with a single variant: `exec`. It also defines a `Backend` tagged union with a single variant: `exec`, which contains a `termio.Exec` instance. There is a corresponding `Config` union for backend configuration.

The backend interface includes methods like `deinit` (cleanup), and the `Backend` union dispatches to the appropriate variant's implementation.

This file matters for the integration because it is where a new backend kind must be added. Today, every Ghostty surface is backed by an `exec` backend, which means it has a child process running in a pseudoterminal. For tmux control mode, surfaces need to be backed by something different: a tmux backend that reads content from the Viewer's Terminal instances and routes keystrokes back through tmux's `send-keys` command. Adding a tmux variant to the `Kind` enum and `Backend` union is one of the fundamental architectural changes needed for the integration.

`src/termio/Exec.zig`

This is the implementation of the `exec` backend, the only backend that exists today. It manages the lifecycle of a child process running inside a pseudoterminal. It handles starting the subprocess, setting up the PTY file descriptors, reading output from the child process, writing input to the child process, and cleaning up when the process exits.

This file matters for the integration not because it needs to be modified, but because it serves as the reference implementation for what a backend looks like. A tmux backend would need to provide the same general interface but with different internals: instead of reading from a PTY file descriptor, it would read from the Viewer's Terminal instances; instead of writing to a PTY, it would send tmux `send-keys` commands.

`src/termio/Termio.zig`

This is the main terminal I/O coordinator. It owns a Backend instance and orchestrates the interaction between the backend, the terminal state (the Terminal struct), the renderer, and the application runtime. It handles threading (there is a dedicated I/O thread), message passing between threads, and the lifecycle of the terminal session.

This file matters for the integration because it is the layer that creates and manages backends. When the application runtime requests a new surface, Termio is what sets up the backend for that surface. For tmux control mode, Termio would need to know how to create a tmux backend instead of an exec backend, and it would need to handle the different lifecycle requirements (a tmux-backed surface does not have its own subprocess, so some of the exec-specific lifecycle management does not apply).

src/termio/message.zig

This file defines the message types used for communication between threads in the terminal I/O system. Messages are how the I/O thread, the renderer thread, and the main application thread communicate without shared mutable state.

This file matters for the integration because tmux control mode introduces new communication requirements. The Viewer runs on the I/O thread of the parent surface (the one where tmux minus-CC was launched), but the tmux-backed child surfaces run on different threads. Getting output data from the Viewer to child surfaces, and getting keystroke data from child surfaces back to the Viewer, will likely require new message types or new uses of existing message types.

Layer 5: Application Runtime (creating and managing GUI elements)

src/apprt/ directory (overview)

The apprtd (application runtime) directory contains the platform-specific GUI framework integration. Ghostty supports multiple platforms: macOS (using AppKit through Swift bindings) and Linux (using GTK). The apprtd layer is responsible for creating windows, tabs, and split views; handling user input events; managing the application lifecycle; and communicating between the GUI and the terminal I/O system.

src/apprt/surface.zig

A Surface in Ghostty represents a single terminal view. It is the thing the user sees and interacts with: it has a visible area where terminal content is rendered, it receives keyboard and mouse input, and it can be part of a tab or a split arrangement. Every Surface currently has a one-to-one relationship with a termio backend: one Surface, one Exec backend, one child process.

This file defines the Surface type and its message passing interface. Surface.Message is used to carry events between the terminal I/O system and the GUI layer. Messages include things like size changes, title updates, clipboard operations, and close requests.

This file matters for the integration because tmux control mode breaks the fundamental assumption that every Surface has its own subprocess. A tmux-backed Surface would not have a child process. Instead, it would get its content from the Viewer's Terminal instance for a specific tmux pane, and it would send input back through the Viewer to tmux. The Surface abstraction may need to be generalized to support this different kind of backing, or a new Surface initialization path may need to be created.

src/apprt/action.zig

This file defines the Action type, which represents user-initiated operations that the application runtime can perform. Actions include things like `new_window` (create a new window), `new_tab` (create a new tab in the current window), `new_split` (create a new split in the current tab), `close_surface` (close the current surface), and many others.

This file matters for the integration because tmux control mode needs to intercept or extend some of these actions. When the user presses the keybinding for new tab while in a tmux control mode session, the application should not create a normal tab with a new subprocess. Instead, it should send a new-window command to tmux, wait for the window-add notification, and then create a native tab backed by the new tmux window. Similarly, `new_split` should become `split-window` in tmux, and `close_surface` should become `kill-pane` or `kill-window`. The action handling layer needs to be aware of whether a surface is tmux-backed and route actions accordingly.

src/apprt/structs.zig

This file defines common data structures used throughout the apprt layer, including types for colors, sizes, positions, and other GUI-related values.

This file may matter for the integration if new struct types are needed to represent tmux-specific GUI state, such as the mapping between tmux window identifiers and native tab identifiers.

src/apprt/gtk.zig and src/apprt/gtk/ directory

These files contain the GTK-specific implementation of the application runtime for Linux. The `gtk.zig` file is the entry point, and the `gtk` subdirectory contains implementations for windows, tabs, splits, surfaces, and other GTK widgets.

These files matter for the integration because any GUI changes need to be implemented for both the macOS and GTK backends. If the integration adds the ability to create tmux-backed surfaces, the GTK implementation needs to support this alongside the macOS implementation. The split view management in particular will need to understand tmux layouts and map them to GTK's pane splitting mechanisms.

src/apprt/embedded.zig

This file provides an embedded application runtime, used when Ghostty is embedded in another application (for example, in an IDE extension). It implements the same interface as the platform-specific runtimes but with a simpler abstraction that delegates window management to the host application.

This file matters for the integration because tmux control mode support should work regardless of which application runtime is active. If the integration changes the apprt interface (for example, by adding new methods for creating tmux-backed surfaces), those changes need to be reflected in the embedded runtime as well.

Layer 6: Terminal Core (the terminal emulator itself)

src/terminal/ directory (relevant core files)

The terminal directory contains the core terminal emulation logic: the VT sequence parser, the screen buffer, cursor management, color handling, and all the terminal state machinery. While most of these files will not need to be modified for the tmux integration, understanding them is important because the Viewer creates Terminal instances for each tmux pane, and these Terminal instances are what would be rendered by tmux-backed surfaces.

The Terminal type (defined in the terminal module, typically accessible through the terminal namespace) is a full terminal emulator state machine. It has a screen (actually multiple screens: primary and alternate), a cursor, mode flags (insert mode, autowrap mode, origin mode, various mouse tracking modes), a scroll region, and a VT parser that processes escape sequences. When the Viewer receives percent-output data from tmux, it decodes the octal escapes and feeds the resulting bytes into a pane's Terminal through its VT stream parser. The Terminal processes those bytes exactly as if they came from a local subprocess, updating its screen buffer, cursor position, and mode flags accordingly.

This matters for the integration because the Terminal instances inside the Viewer's Pane structs are the data source for rendering. A tmux-backed surface would need to read from one of these Terminal instances rather than from a Terminal that is connected to a local PTY. The rendering pipeline does not care where the Terminal's data comes from; it just reads the screen buffer and draws it. The challenge is in the plumbing: getting the right Terminal connected to the right Surface.

Summary: How These Files Relate to Each Other

The data flow through these files forms a pipeline. Bytes from the tmux process enter through `dcs.zig`, which detects the DCS control mode sequence. Those bytes flow into `control.zig`, which parses them into Notification values. Those Notification values flow into `stream_handler.zig`, which feeds them to the Viewer in `viewer.zig`. The Viewer processes the notifications, maintains internal state (using `layout.zig` and `output.zig` as sub-parsers), and emits Action values back to the stream handler. The stream handler processes command actions by sending strings back to tmux through the I/O system. But for windows actions, the stream handler does nothing, and this is where the pipeline breaks.

To complete the integration, the stream handler would need to take windows actions and communicate them to the apprt layer (through `surface.zig` and `action.zig`), which would create or update native tabs and splits. Each of those native GUI elements would be a Surface backed by a new kind of Backend (defined in `backend.zig`), which reads from the Viewer's Terminal instances rather than from a subprocess PTY.

The build configuration (`build_options.zig`), module wiring (`tmux.zig`, `main.zig`), and the Exec backend (`Exec.zig`) serve as supporting infrastructure: the first two control whether the feature is compiled in and how types are exported, and the last provides the template for what a new tmux backend would look like.